Task Manager REST API

Project brief — your first async Rust backend

What you're building

A JSON REST API that manages tasks — create, read, update, delete. It runs as an HTTP server on localhost, stores data in SQLite, and handles everything asynchronously with Tokio. This is the standard backend pattern used in real production services.

Endpoints

Method	Route	What it does	Body / Params
`POST`	`/tasks`	Create a new task	JSON body
`GET`	`/tasks`	Return all tasks	—
`GET`	`/tasks/:id`	Return one task by ID	Path param :id
`PUT`	`/tasks/:id`	Update title / status	JSON body + :id
`DELETE`	`/tasks/:id`	Delete a task	Path param :id

Data model

One struct, one table.

```
// models.rs
struct Task {
id: i64,
title: String,
done: bool, // false on creation
created_at: String, // ISO timestamp
}
```

```
-- migrations/001_tasks.sql
CREATE TABLE IF NOT EXISTS tasks (
id INTEGER PRIMARY KEY AUTOINCREMENT,
title TEXT NOT NULL,
done BOOLEAN NOT NULL DEFAULT 0,
created_at TEXT NOT NULL
);
```

File structure

File	Responsibility

`src/main.rs`	Start Tokio runtime, create DB pool, build Axum router, bind to port.
`src/models.rs`	Task struct. Derive Serialize, Deserialize. CreateTask input struct.
`src/db.rs`	init_db() — run migration. Return SqlitePool shared across handlers.
`src/handlers.rs`	One async fn per endpoint. Each receives State(pool) + optional Json/Path.
`src/errors.rs`	AppError enum using thiserror. impl IntoResponse to return HTTP status codes.
`migrations/001.sql`	CREATE TABLE tasks — run once on startup via db::init_db().
`.env`	DATABASE_URL=sqlite://tasks.db

Tech stack

Crate	Why	New concept
`axum`	HTTP framework — routing, extractors, middleware	Router, State, Json extractor
`tokio`	Async runtime — runs your async fns	#[tokio::main], .await
`sqlx`	Async DB queries, compile-time checked SQL	query!, query_as!, Pool
`serde`	JSON body in / out	Serialize + Deserialize
`thiserror`	Custom error type that converts to HTTP response	#[derive(Error)], IntoResponse
`dotenvy`	Load DATABASE_URL from .env file	dotenvy::dotenv()

Cargo.toml snippet — sqlx needs the sqlite and runtime-tokio features explicitly:

```
axum = "0.7"
tokio = { version = "1", features = ["full"] }
sqlx = { version = "0.7", features = ["sqlite", "runtime-tokio", "chrono"] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
thiserror = "1"
dotenvy = "0.15"
```

Build order — do it in this exact sequence

1 **Scaffold + compile**
cargo new task-manager. Add all deps. Write a bare #[tokio::main] async fn main(). Make it compile before touching anything else.

2 **models.rs**
Define Task struct and a separate CreateTaskInput struct (only title field — no id or created_at, those are set by the server).

3 **db.rs**
Write init_db(): connect to SQLite, run the CREATE TABLE migration, return SqlitePool. Call it from main and confirm the .db file is created.

4 **handlers.rs — GET /tasks first**
Wire up one route. Return an empty JSON array. Test it with curl. Add handlers one at a time, test each before writing the next.

5 **errors.rs**
Define AppError with thiserror. Implement IntoResponse. Replace all .unwrap() in handlers with proper ? error propagation.

6	**Polish + README** Add .env, replace hardcoded strings. Test all 5 endpoints with curl. Write GitHub README. Done.

New things vs the expense tracker

Expense tracker	Task Manager API
Sync functions	async fn + .await everywhere
std::fs for storage	sqlx pool + SQL queries
Clap CLI args	Axum extractors (Json, Path, State)
println! output	HTTP JSON responses
No error type	thiserror + IntoResponse

Ground rules

— Build and test one endpoint at a time. curl localhost:3000/tasks after every handler you write.

— Never use .unwrap() in a handler. Use ? and your AppError type — this is the whole point of errors.rs.

— Don't add auth, pagination, or filtering until all 5 basic endpoints work cleanly.

— Commit after each working step. Dead code and broken experiments go in separate branches.

— Target: done in 5 to 7 days. This project on your GitHub + the expense tracker = a convincing Rust portfolio.

You already know how to build Rust projects. This is the same thing with async and HTTP added.